ROAR Solo Mission

Technical Documentation

Part 1

Prepared by:

Saif Elden Mahmoud

Date:

April 26, 2026

Contents

1 Autonomous Systems Fundamentals **3**
1.1 Q1: What does it mean for a system to be autonomous? 3
1.2 Q2: Role of simulation and testing 3
1.3 Q3: Limitations of simulation environments 4
1.4 Q4: What it means for a sensor model to be unrealistic 5

2 ROS 2 Fundamentals **6**
2.1 Q1: ROS 2 concepts 6
2.2 Q2: Quality of Service (QoS) Policies 6

3 Path Planning **7**
3.1 Q1: Path Planning Definition 7
3.2 Q2: Global vs. Local Planners 7
3.3 Q3: Occupancy vs. Height Maps 8
3.4 Q4: A* Algorithm 8
3.5 Q5: Artificial Potential Field (APF) 9

4 Control **10**
4.1 Q1: Define Control Theory 10
4.2 Q2: Open-loop vs. Closed-loop Control 11
4.3 Q3: Controller Stability 11
4.4 Q4: Pure Pursuit Algorithm 12
4.5 Q5: Standard vs. Adaptive Pure Pursuit 13

5 Localization **14**
5.1 Q1: Sensor Fusion Concept 14
5.2 Q2: IMU and Drift 14
5.3 Q3: Kalman vs. Extended Kalman Filter 14

6 Perception **15**
6.1 Q1: LiDAR vs. Cameras 15
6.2 Q2: Monocular vs. Stereo Cameras 16
6.3 Q3: ArUco Markers 17

List of Figures

1 Sensor noise and perception artifacts in simulated vs. real environments . 4
2 Deterministic simulation behavior vs. stochastic real-world conditions . . 5
3 Comparison of occupancy grid maps and height maps for terrain representation 8
4 Artificial Potential Field concept showing attractive and repulsive forces . 9
5 Control theory framework translating commands to actuator signals . . . 10
6 Open-loop control (blind) vs. closed-loop control (feedback-based) 11
7 Stable vs. unstable controller behavior over time 12
8 Pure Pursuit algorithm geometry showing lookahead point and circular arc 12
9 Adaptive Pure Pursuit adjusting lookahead distance based on velocity . . 13
10 LiDAR (active sensing) vs. Camera (passive sensing) comparison 15
11 Monocular (single) vs. Stereo (dual) camera depth estimation 16
12 ArUco marker detection and 6D pose estimation for robotic applications 17

1 Autonomous Systems Fundamentals

1.1 Q1: What does it mean for a system to be autonomous?

An autonomous system perceives its environment, processes information, and executes actions to achieve goals without continuous human intervention. Unlike automated systems that follow fixed rules, autonomous systems adapt to dynamic and unpredictable conditions.

Core Architecture: Sense-Think-Act Loop

The system operates through four interconnected modules:

1. **Perception** — Processes sensor data (LiDAR, cameras, RADAR) to detect and classify objects, producing an environmental map for planning.
2. **Localization** — Determines the system's precise position and orientation by comparing sensor data against known maps and integrating motion data (GPS, IMU).
3. **Planning** — The decision-making core that generates optimal trajectories using the environmental map and current position. Includes global route planning and local obstacle avoidance.
4. **Control** — Translates planned trajectories into actuator commands (steering, throttle, braking) to execute the desired motion.

Data Flow: Sensors → Perception & Localization → Planning → Control → Actuators. This cycle repeats continuously at high frequency, enabling real-time adaptation.

1.2 Q2: Role of simulation and testing

Simulation and testing provide a continuous feedback loop that bridges conceptual design and safe physical deployment.

1. **System Design (Virtual Prototyping)** — Enables testing of different sensor configurations and algorithms without physical hardware costs. Design flaws are identified early through rapid software iteration.
2. **Validation** — Two-stage verification process:
   - *Software-in-the-Loop (SIL):* Tests core logic in a purely virtual environment to verify correct code execution.
   - *Hardware-in-the-Loop (HIL):* Runs code on actual microcontrollers while simulating the physical world, validating real-time processing capabilities and communication protocols (CAN bus, I2C).
3. **Risk Reduction** — Safely tests dangerous edge cases (sensor failures, extreme weather, unexpected obstacles) that are impractical or unsafe to test physically. Automated regression testing ensures code updates don't break existing functionality.

1.3 Q3: Limitations of simulation environments

Simulation cannot perfectly replicate reality due to the "Sim-to-Real" gap, caused by three technical discrepancies:

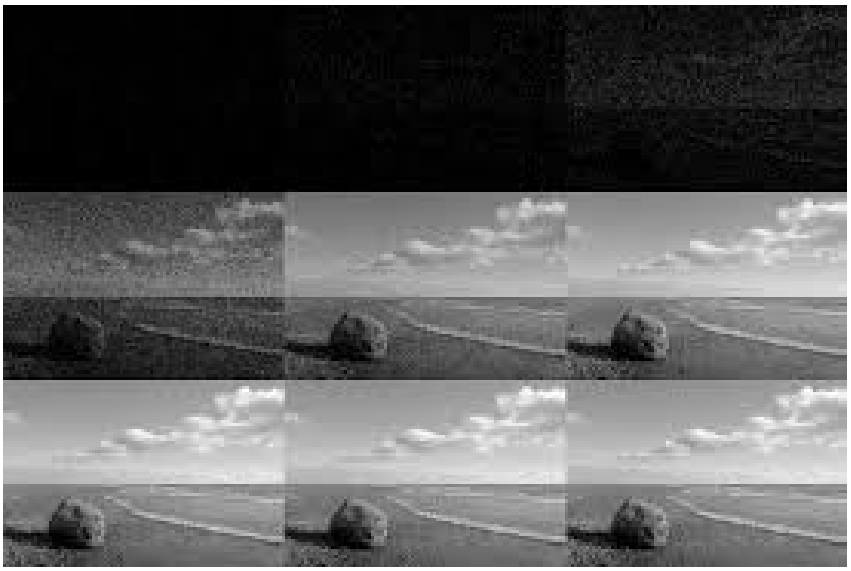


Figure 1: Sensor noise and perception artifacts in simulated vs. real environments

1. **Physics Simplification** — Simulators use idealized models for real-time performance:
   - *Non-linear Dynamics:* Real friction, tire slip, and aerodynamics vary with temperature and wear; simulators often use linear approximations.
   - *Deformable Objects:* Accurate modeling (e.g., tire deformation, cable flex) requires computationally expensive finite element analysis, typically omitted.
   - *Discrete Time Steps:* Physics calculated at intervals (e.g., 1ms) can miss fast collisions or become unstable, unlike continuous real-world physics.
2. **Sensor Noise and Perception Artifacts** — Simulated sensors are idealized unless explicitly degraded:
   - *Idealized Data:* Simulated LiDAR produces clean point clouds; real LiDAR suffers from multi-path reflections, ghosting in rain/fog, and infrared interference.
   - *Material Properties:* Simulators struggle with material-specific reflections (e.g., matte black surfaces may be invisible to real LiDAR but visible in simulation).
   - *Lens Effects:* Real cameras exhibit chromatic aberration and rolling shutter effects that are expensive to simulate accurately.
3. **Timing and Asynchronicity** — Simulators control time; reality does not:
   - *Jitter and Latency:* Simulated data exchange is instant; real systems have bus delays, contention, and electromagnetic interference causing stale data.

- *Race Conditions:* Real hardware uses independent clocks, leading to dropped frames or delayed responses when sensors fire during CPU-intensive operations—scenarios synchronized simulations never encounter.

1.4 Q4: What it means for a sensor model to be unrealistic

In simulators like Gazebo, unrealistic sensor models provide "perfect" data that ignores physical and electrical imperfections of real hardware, creating a Sim-to-Real gap where systems perform flawlessly in simulation but fail in reality.

1. **Inaccurate Noise and Bias** — Simulators use simple Gaussian noise or none at all. Real sensors have bias (drifting constant offset) and non-linear noise. Control loops tuned for clean simulated data become unstable or jittery with real sensor signals.

2. **Fixed vs. Variable Update Rates** — Simulators provide data at perfectly steady frequencies (e.g., exactly 50Hz). Real sensors experience latency and jitter from bus congestion (CAN, I2C). Timing-dependent algorithms (EKF, high-speed PID) fail when data arrives stale or irregularly.

3. **Lack of Environmental Interaction** — Simulators treat surfaces as simple collision boxes. Real material properties matter: mirrors/windows may be invisible to LiDAR; dark carpets absorb IR beams. Perception modules fail to detect obstacles that were easily visible in simplified virtual environments.

4. **Deterministic vs. Stochastic Behavior** — Simulators are deterministic (same input = same output). Real environments are stochastic with random interference (sunlight washing out cameras, electromagnetic interference from motors). Systems not stress-tested with random interference become fragile when deployed.

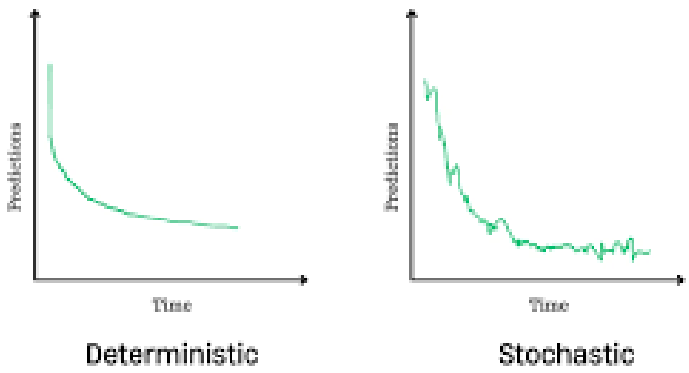


Figure 2: Deterministic simulation behavior vs. stochastic real-world conditions

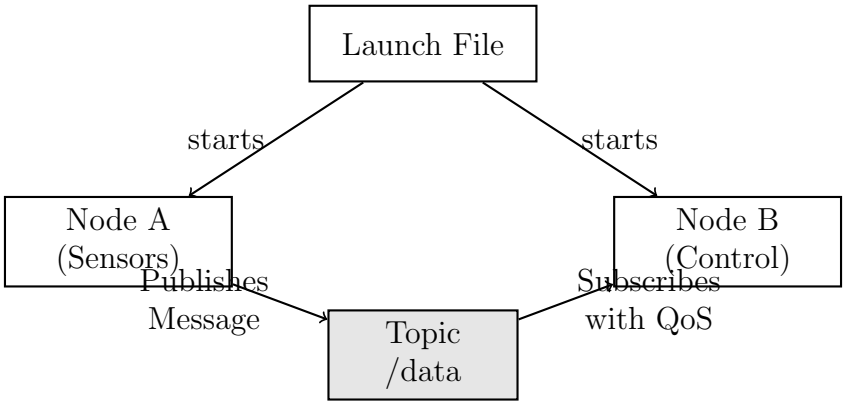
2 ROS 2 Fundamentals

2.1 Q1: ROS 2 concepts

In ROS 2, a robotic system is a collection of independent processes called Nodes that communicate using a Publish-Subscribe model.
Core Components:

- **Node** — Single-purpose executable (e.g., camera node, motor node).
- **Topic** — Named communication channel or "bus."
- **Message** — Data structure (e.g., Int32, Pose) sent over topics.
- **Publisher** — Node that sends data onto a topic.
- **Subscriber** — Node that listens for data on a topic.
- **Launch File** — Script (Python, XML, YAML) that starts multiple nodes with specific configurations.

System Architecture Visualization:



2.2 Q2: Quality of Service (QoS) Policies

QoS policies tune node communication based on data requirements (e.g., "receive every packet" vs. "only latest data").
Key Policies:

1. **Reliability**
   - *Reliable:* Lost messages are resent (critical commands like emergency stop).
   - *Best Effort:* Messages sent once; losses are not recovered (high-frequency sensor data like LiDAR).

2. **Durability**

   - *Transient Local:* Publisher saves recent messages; late subscribers receive past messages (static data like map metadata).
   - *Volatile:* Late subscribers only see messages sent after connection.

QoS Mismatch: Connections form only if Publisher and Subscriber are compatible. The Subscriber must be at least as demanding as the Publisher.

Failure Scenario: If Publisher uses Best Effort but Subscriber demands Reliable, the connection fails silently—the subscriber requires guarantees the publisher cannot provide, preventing message exchange.

---

3 Path Planning

3.1 Q1: Path Planning Definition

Path planning finds a geometric path from start to goal, bridging perception and action in the navigation stack. The robot must satisfy two constraint types:

- **Environmental Constraints** — Collision-free paths accounting for static and dynamic obstacles.
- **Kinematic Constraints** — Paths respecting physical limits (turning radius, maximum velocity). Example: car-like robots cannot move sideways.

3.2 Q2: Global vs. Local Planners

These planners work hierarchically to balance long-term goals with immediate safety:

- **Global Planner** — Operates on the entire known map to find the most efficient route. Uses static information (pre-loaded occupancy grid) and calculates infrequently.
- **Local Planner** — Focuses on a small window around the robot. Uses real-time sensor data (LiDAR, depth cameras) to detect dynamic obstacles and generates velocity commands.

Interaction: Global Planner provides waypoints; Local Planner follows them while avoiding obstacles not on the original map.

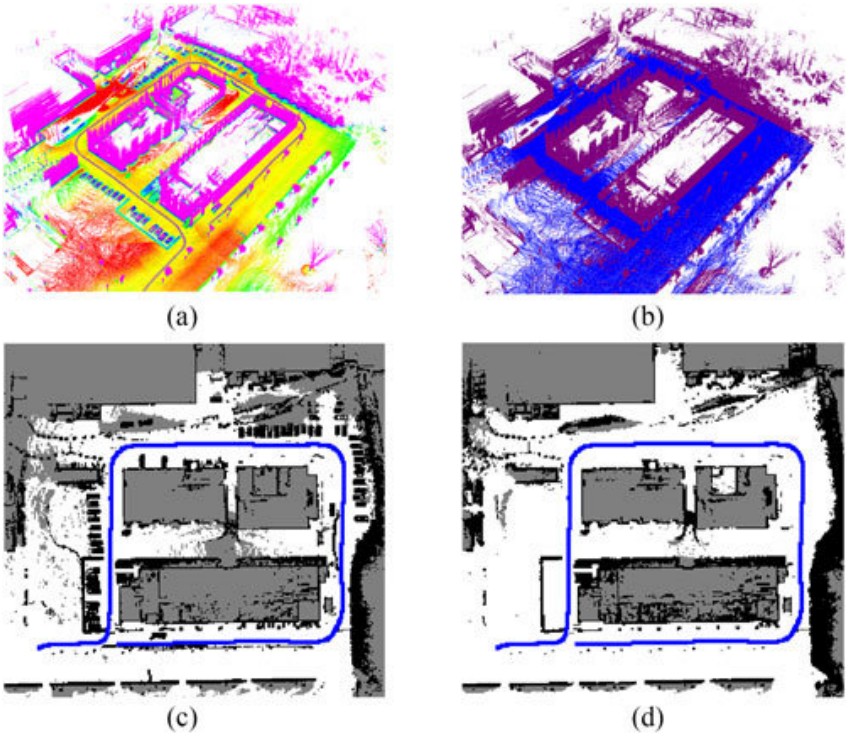


Figure 3: Comparison of occupancy grid maps and height maps for terrain representation

3.3 Q3: Occupancy vs. Height Maps

Environment representation depends on terrain complexity:

- **Occupancy Grid Maps** — Discretize space into cells marked as "free," "occupied," or "unknown."
  - *Advantage:* Simple, memory-efficient for 2D navigation.
  - *Use Case:* Indoor mobile robots on flat floors.
- **Height Maps (Elevation Maps)** — Store height (z) value for every (x,y) coordinate.
  - *Advantage:* Captures 3D geometry (slopes, curbs, steps) without full point cloud memory cost.
  - *Use Case:* Outdoor rovers or legged robots on uneven terrain.

3.4 Q4: A* Algorithm

A* is a heuristic-based search algorithm that finds the least-cost path by evaluating nodes using:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ — Actual cost from start node to current node n
- $h(n)$ — Heuristic estimate of cost from n to goal (e.g., Euclidean distance)

Optimality Guarantee: A* finds the optimal path if the heuristic is:

- *Admissible:* Never overestimates true cost to goal
- *Consistent:* Satisfies triangle inequality

Comparison with Dijkstra's: Dijkstra's is A* with $h(n) = 0$. It explores all directions equally (breadth-first), always finding the shortest path but much slower than A* because it lacks directional guidance toward the goal.

3.5 Q5: Artificial Potential Field (APF)

APF treats the robot as a particle in a virtual magnetic field, moving according to the gradient of a potential function combining two forces:

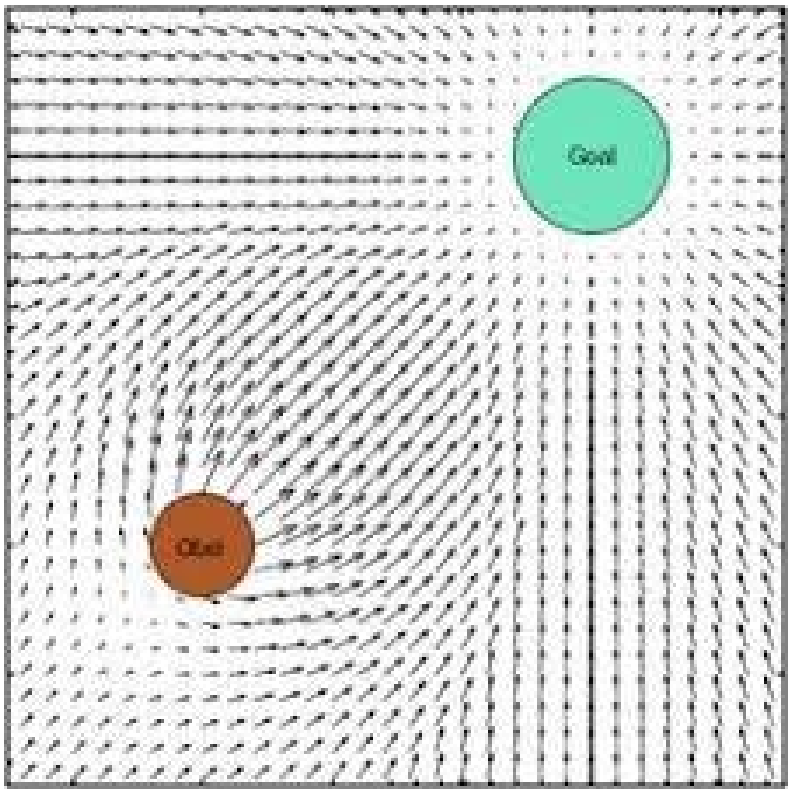


Figure 4: Artificial Potential Field concept showing attractive and repulsive forces

- **Attractive Force (F_{att})** — Pulls robot toward goal; strength increases with distance from target.
- **Repulsive Force (F_{rep})** — Pushes robot away from obstacles; very strong when close, drops to zero outside safety buffer.

The robot follows the resultant force: $F_{total} = F_{att} + F_{rep}$, sliding down the potential gradient toward the goal.

Mathematical Formulation:

Total potential field: $U(q) = U_{att}(q) + U_{rep}(q)$

Resulting force: $F(q) = -\nabla U(q)$

Where U_{att} is typically quadratic: $\frac{1}{2}\xi\rho^2(q, q_{goal})$, and U_{rep} is inversely proportional to obstacle distance.

Main Limitations:

1. **Local Minima** — Forces cancel before reaching goal (e.g., U-shaped obstacles), causing robot to get stuck.
2. **Oscillations** — Rapid bouncing between repulsive forces in narrow passages leads to unstable movement.
3. **GNRON (Goal Non-Reachable with Obstacles Nearby)** — Obstacles very close to goal prevent robot from reaching target due to strong repulsive forces.

---

4 Control

4.1 Q1: Define Control Theory

Control theory provides the mathematical framework that translates high-level commands ("go there") into precise actuator signals ("send 3.3V to steering motor").

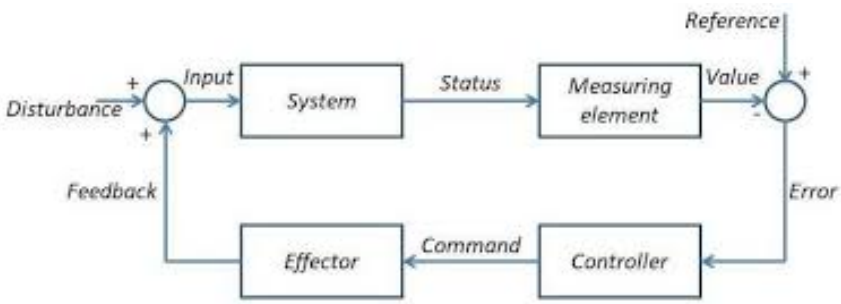


Figure 5: Control theory framework translating commands to actuator signals

Purpose: Manages physical system behavior to execute commands smoothly and accurately.

Challenge: Forces hardware to follow software commands while actively compensating for real-world physics (momentum, friction, wind resistance, sensor noise).

Key Components:

- *Plant:* The physical system being controlled (vehicle, robot arm)
- *Controller:* Algorithm computing control signals

- *Actuators:* Hardware executing commands (motors, servos)
- *Sensors:* Measuring system state for feedback

4.2 Q2: Open-loop vs. Closed-loop Control

The fundamental distinction: checking your work versus hoping for the best.

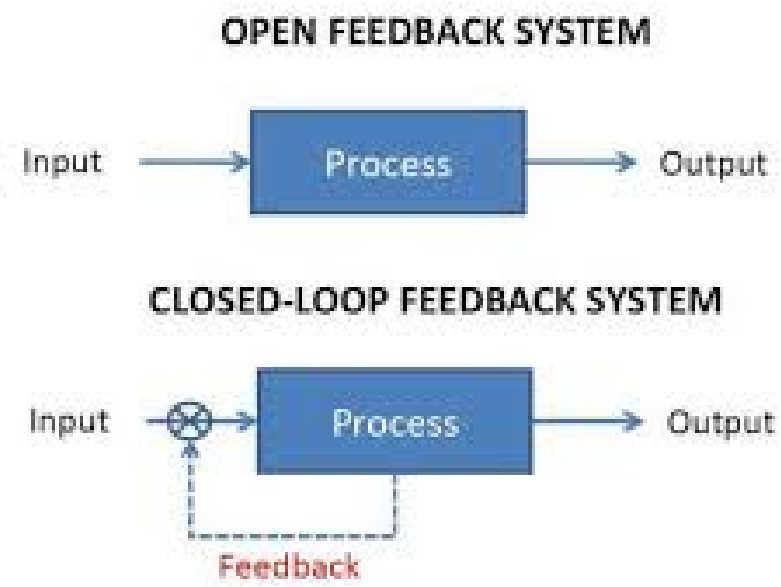


Figure 6: Open-loop control (blind) vs. closed-loop control (feedback-based)

- **Open-loop (Blind)** — Sends command without verification (e.g., "spin tires for 3 seconds"). If vehicle slips on ice, system remains unaware it didn't move. Simple but unreliable in unpredictable environments.
- **Closed-loop (Aware)** — Uses sensor feedback to continuously measure error (desired state - actual state) and adjust control signals. Robust to disturbances but requires accurate sensors and careful tuning.

Example: Open-loop cruise control maintains fixed throttle; closed-loop adjusts throttle based on actual speed measurement.

4.3 Q3: Controller Stability

Stability determines whether a system behaves predictably or catastrophically.

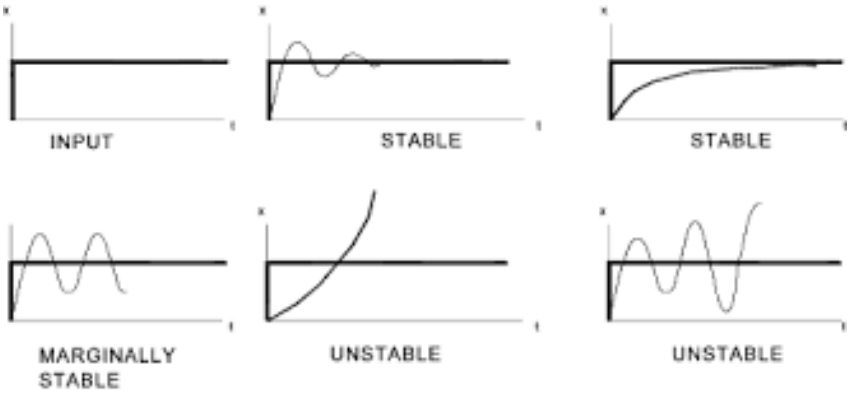


Figure 7: Stable vs. unstable controller behavior over time

- **Stable Behavior** — System returns to desired state after disturbances. If wind pushes vehicle off course, controller gently steers back without overshooting.
- **Unstable Behavior** — System overreacts to errors, causing oscillations that grow over time. Small steering error triggers excessive correction, leading to violent swerving and potential crash.

Mathematical Criterion: A system is stable if all poles of its transfer function have negative real parts (for continuous systems) or magnitude less than 1 (for discrete systems).

4.4 Q4: Pure Pursuit Algorithm

Pure Pursuit mimics how a dog chases a rabbit—aiming for where the target will be, not where it is.

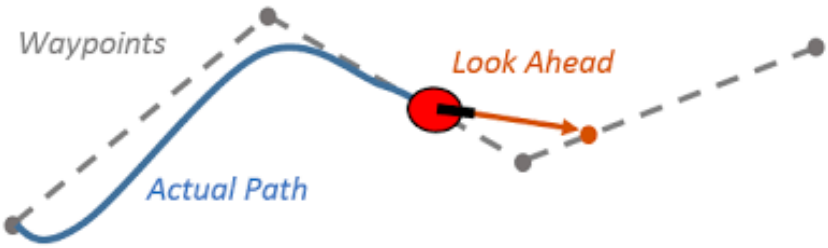


Figure 8: Pure Pursuit algorithm geometry showing lookahead point and circular arc

Concept: Algorithm selects a "lookahead point" at fixed distance ahead on desired path.
Geometry: Calculates circular arc connecting vehicle's rear axle to lookahead point. The arc radius determines required steering angle.
Output: Steering command: $\delta = \arctan\left(\frac{2L\sin(\alpha)}{l_d}\right)$

Where L is wheelbase, α is angle to lookahead point, l_d is lookahead distance.
Advantage: Geometrically simple, computationally efficient, naturally smooth trajectories.

4.5 Q5: Standard vs. Adaptive Pure Pursuit

Lookahead distance dramatically affects performance at different speeds.

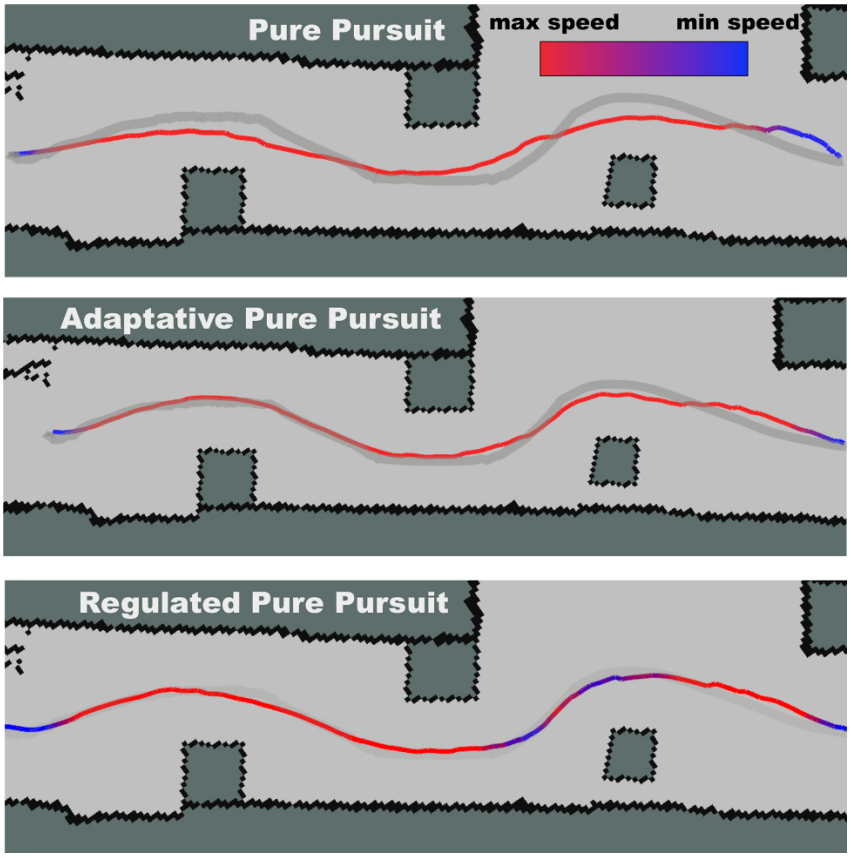


Figure 9: Adaptive Pure Pursuit adjusting lookahead distance based on velocity

- **Standard Pure Pursuit** — Fixed lookahead distance. Optimal at one speed but problematic at others. Short distance at high speed causes aggressive swerving; long distance at low speed results in wide corner cutting.
- **Adaptive Pure Pursuit** — Dynamically adjusts lookahead: $l_d = k_v \cdot v + l_{min}$, where k_v is velocity gain and l_{min} is minimum distance. Stretches lookahead at high speeds (smooth lane changes) and shrinks at low speeds (tight cornering).

Benefit: Prevents high-speed instability while maintaining low-speed precision, adapting to vehicle dynamics across operating range.

5 Localization

5.1 Q1: Sensor Fusion Concept

Sensor fusion mathematically blends data from multiple sensors to create a single, highly accurate world representation—like a team where each member has unique strengths covering others' weaknesses.
Why Necessary: No single sensor is perfect:

- *Cameras:* Excellent for lane detection but blind at night or in direct sunlight
- *LiDAR:* Performs well in darkness but confused by rain or fog
- *GPS:* Provides global position but fails in tunnels or urban canyons

Result: When one sensor fails or becomes noisy, others compensate instantly, preventing system failure. Fusion algorithms weight sensor contributions based on current reliability, creating robust state estimates.

5.2 Q2: IMU and Drift

An Inertial Measurement Unit (IMU) measures motion without external references—like closing your eyes in a moving car and sensing acceleration and turns.
Structure: Packages three sensors on one chip:

- *Accelerometer:* Measures linear forces and gravity
- *Gyroscope:* Measures rotation rates
- *Magnetometer:* Digital compass (optional)

Role in Localization: GPS updates slowly (∼10 Hz); IMU provides high-frequency updates (100+ Hz) to estimate pose between GPS measurements.
The Flaw (Drift): Position requires double integration of acceleration. Sensor noise, though small, accumulates rapidly through integration. After seconds, drift occurs—the system believes it moved meters while actually stationary. Fusion with absolute position sensors (GPS, visual landmarks) corrects this drift.

5.3 Q3: Kalman vs. Extended Kalman Filter

These algorithms balance model predictions with noisy sensor measurements—the ultimate "trust but verify" approach.

- **Standard Kalman Filter (KF)** — Optimal for linear systems where relationships scale in straight lines.
  - *How it works:* Recursively estimates true state by minimizing mean squared error
  - *When to use:* Simple 1D problems (temperature tracking, straight-line motion)
  - *Limitation:* Assumes linear dynamics and measurement models

- **Extended Kalman Filter (EKF)** — Handles non-linear systems using linearization.
  - *How it works:* Uses Jacobian matrices (calculus) to locally linearize non-linear functions (sine, cosine in rotation) at each time step
  - *When to use:* Modern robotics with steering, rotation, and multiple sensors
  - *Advantage:* Handles realistic vehicle dynamics (turning arcs, sensor rotations)

Key Difference: KF assumes $x_{k+1} = Ax_k + Bu_k$; EKF linearizes $x_{k+1} = f(x_k, u_k)$ using $\frac{\partial f}{\partial x}$.

---

6 Perception

6.1 Q1: LiDAR vs. Cameras

The fundamental difference: feeling your way with a precise tape measure versus looking around with your eyes.

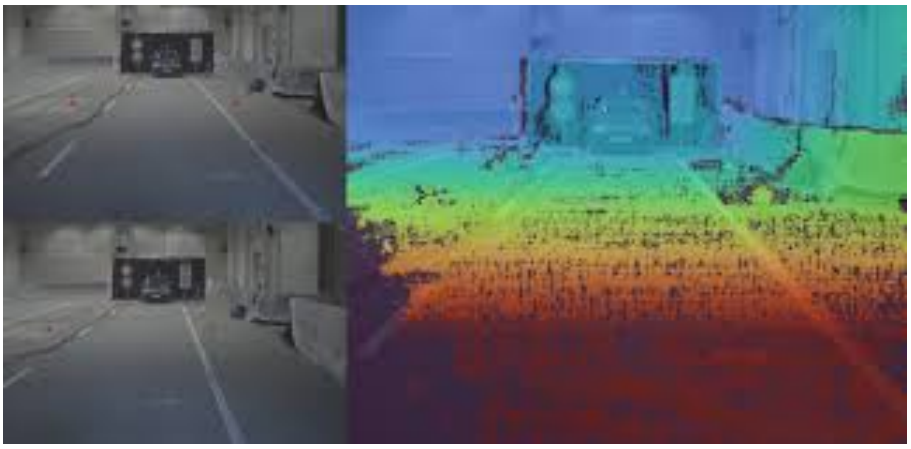


Figure 10: LiDAR (active sensing) vs. Camera (passive sensing) comparison

How They Work:

- **LiDAR (Active)** — Emits millions of laser pulses per second, measuring Time-of-Flight (how long light takes to bounce back).
- **Camera (Passive)** — Captures ambient light (photons) bouncing off objects onto its image sensor.

Data Output:

- *LiDAR:* 3D Point Cloud with exact (X, Y, Z) coordinates. Inherently knows distance but is colorblind.
- *Camera:* 2D Pixel Grid (RGB image). Knows color and texture but has no native depth information.

Processing Requirements:

- *LiDAR:* Low processing load for obstacle detection. Simple rule-based logic: "if point < 1m, brake."
- *Camera:* High processing load. Requires heavy AI/Neural Networks to distinguish pedestrians from painted signs.

Failure Conditions:

- *LiDAR:* Laser scatter in heavy rain, fog, snow, or reflective surfaces (glass, mirrors).
- *Camera:* Poor lighting—pitch black, blinding sunlight glare, rapid transitions (tunnel exit).

6.2 Q2: Monocular vs. Stereo Cameras

The difference between covering one eye versus using both.

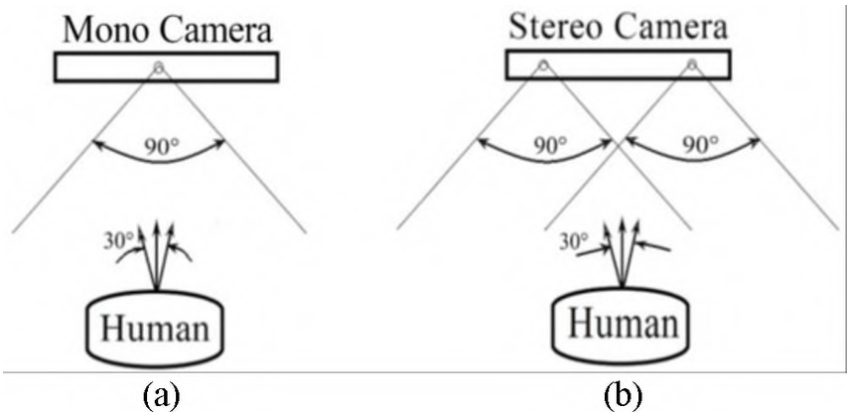


Figure 11: Monocular (single) vs. Stereo (dual) camera depth estimation

- **Monocular (Single Camera)**
  - *Depth Estimation:* No native depth perception. Uses contextual tricks—if a known 75cm stop sign appears small, AI infers it's far away.
  - *Complexity:* Computationally cheap, inexpensive hardware. Easily runs on Raspberry Pi.
  - *Limitation:* Depth estimates are unreliable for unknown objects.
- **Stereo (Dual Cameras)**
  - *Depth Estimation:* Works like human eyes. Views same object from two angles, measures horizontal disparity (pixel shift between left/right images) to calculate exact distance.

- *Complexity:* Computationally heavy. Must process two HD streams, match corresponding pixels in real-time, and compute depth for every pixel.
- *Advantage:* Accurate depth for any object, not just known ones.

Mathematical Basis: Stereo depth: $Z = \frac{f \cdot B}{d}$, where f is focal length, B is baseline (camera separation), d is disparity.

6.3 Q3: ArUco Markers

Simplified, ruggedized QR codes designed for robotic vision—readable from across a room.

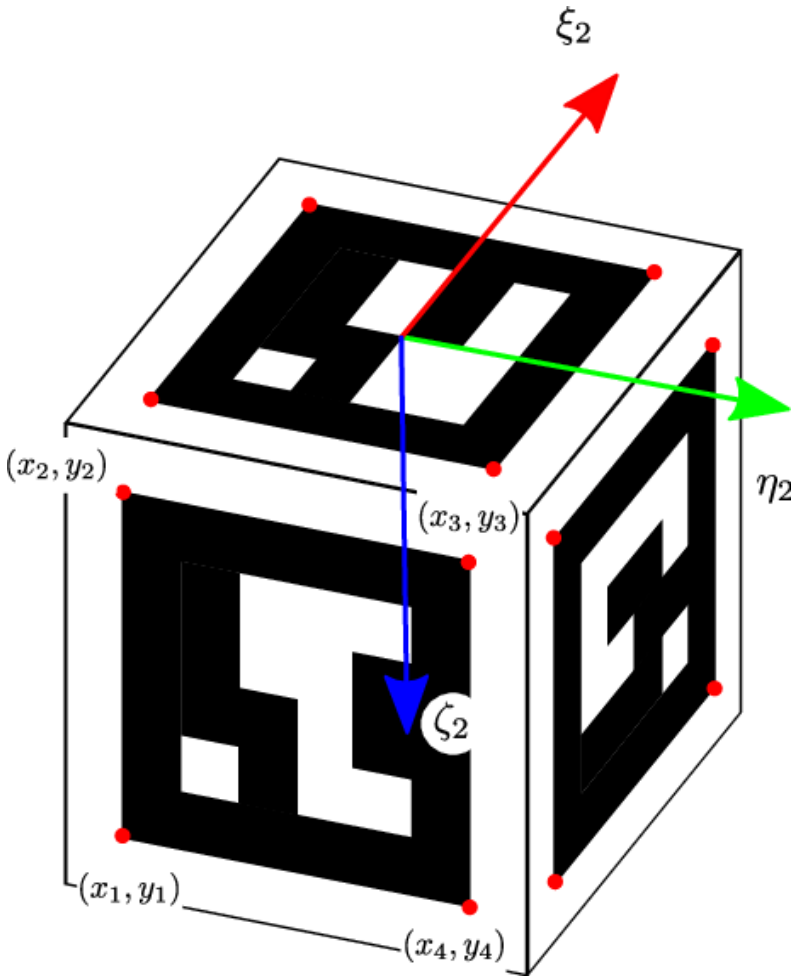


Figure 12: ArUco marker detection and 6D pose estimation for robotic applications

Structure & Detection: High-contrast black-and-white square grids. Camera algorithm converts image to grayscale, detects stark black square outlines, extracts inner binary pattern, and matches against known dictionary to identify valid markers.
Information Extracted:

- *Marker ID:* Identifies which specific marker (e.g., "Marker #42")

- *6D Pose:* Calculates position (X, Y, Z) and orientation (Roll, Pitch, Yaw). Since markers are perfect flat squares, any skewed trapezoid appearance reveals exact 3D pose through perspective geometry.

Applications:

1. **Localization** — Place Marker #1 on a wall. When rover detects it, instantly calculates its exact room position relative to the wall.

2. **Manipulation** — Attach marker to tool or box. Robotic arm uses 6D pose to angle gripper perfectly for precise grasping.

Advantage: Provides absolute position reference without GPS, enabling precise indoor navigation and object manipulation.